Universidade Estadual de Campinas
Instituto de Computação

Tarefa 5

Composição, Fases de Construção e Acoplamento

MC322 - Programação Orientada a Objetos

Professor: Marcelo da Silva Reis
PEDs: Athyrson Ribeiro, Matheus Lozano
PADs: João Pedro Frare

---

1 Descrição Geral

Ao longo das tarefas anteriores, a nossa planta industrial ganhou estratégias dinâmicas (Strategy), segurança de tipos (Enums) e um sistema de *build* moderno (Gradle). Contudo, até o momento, a principal entidade transformadora da sua fábrica (seja ela uma Máquina Mecânica, um Forno de Padaria ou um Caldeirão Mágico) tem sido modelada como um objeto monolítico.

Em uma planta industrial, uma unidade produtiva raramente é um objeto isolado. Uma máquina, por exemplo, pode ser formada por diferentes componentes, como motores, sensores, atuadores e controladores. Esses componentes possuem responsabilidades próprias, mas podem fazer parte da estrutura interna da máquina e ter seu ciclo de vida controlado por ela.

Nesta **Tarefa 5**, refatoraremos a arquitetura do sistema introduzindo **Relações de Composição** (relacionamentos estruturais "Todo-Parte"). Como o tema do projeto é **livre**, vocês deverão identificar quais são as partes menores que formam a entidade principal da sua fábrica. O objetivo é dividir a complexidade, gerenciando cuidadosamente o ciclo de vida dos objetos (fases de construção), a multiplicidade das relações (cardinalidade) e garantindo um encapsulamento estrito através da **Lei de Deméter**.

O objetivo não é simplesmente adicionar atributos que apontem para outras classes, mas compreender quando um objeto deve ser responsável por outro objeto, como esse relacionamento afeta o ciclo de vida das instâncias e como manter o encapsulamento entre os diferentes níveis do sistema.

A tarefa também introduzirá o uso de **diagramas UML** como ferramenta de modelagem, permitindo representar visualmente as relações entre o Agrupador, o Todo e suas Partes.

1.1 Evolução da Planta

Independentemente do tema escolhido, a planta agora deve:

- decompor uma unidade produtiva em componentes menores;
- representar relações de composição entre uma entidade (o Todo) e suas Partes;
- representar uma relação de agrupamento entre várias unidades produtivas;
- utilizar pelo menos uma relação com cardinalidade 1:1 e uma relação com cardinalidade 1:N;
- controlar adequadamente o ciclo de vida das Partes que pertencem a um Todo;
- impedir que detalhes internos da composição sejam expostos desnecessariamente;
- utilizar métodos de delegação para interagir com as Partes;
- proteger coleções internas contra modificações externas;
- propagar comportamentos relevantes pela estrutura composta.

2 Objetivos

- Compreender e aplicar relacionamentos estruturais de **Composição Estrita** (onde a Parte não tem sentido ou existência independente fora do escopo do Todo);

- Controlar as **Fases de Construção**, garantindo que o Todo seja inicializado instanciando e configurando suas Partes internamente;
- Modelar **Cardinalidades** diferentes (1:1 e 1:N) utilizando estruturas de dados adequadas (`List`, `Set`);
- Exercitar o **Princípio de Deméter (Desacoplamento)**, estruturando fachadas (*Facades*) internas para que os clientes do sistema desconheçam as subpartes;
- Representar relações de composição e agregação utilizando UML.

3 Atividades

3.1 Modelagem da Composição (O Todo e as Partes)

A dupla deverá identificar a classe que representa a **unidade produtiva individual** da sua fábrica (referida aqui genericamente como "O Todo"). Vocês deverão decompor esse objeto criando **pelo menos três novas classes** que representem seus subcomponentes internos ("As Partes").

As Partes devem representar componentes que façam sentido no domínio escolhido e deverão possuir responsabilidades próprias.

Cada Parte deverá possuir, quando aplicável:

- atributos;
- estado;
- métodos;
- regras de funcionamento próprias.

Exemplos de Temas vs. Modelagem:

- **Fábrica de Carros:** O Todo é o `RoboDeMontagem`. As Partes podem ser `BracoArticulado`, `SensorDeSolda` e `PlacaDeRede`.
- **Padaria Mágica:** O Todo é o `FornoEncantado`. As Partes podem ser `CristalDeAquecimento`, `TermostatoArcano` e `EsteiraDeAssadeiras`.
- **Fábrica de Software:** O Todo é o `ServidorDeBuild`. As Partes podem ser `Compilador`, `AnalisadorDeQualidade` e `GerenciadorDeMemoria`.

Os exemplos são apenas ilustrativos. A modelagem deverá ser adaptada ao tema escolhido pela dupla.

3.2 Composição e Ciclo de Vida

A relação entre o Todo e suas Partes deverá representar uma **composição**.

A **composição estrita** exige exclusividade sobre o ciclo de vida. Isso significa que as instâncias das Partes devem ser criadas **obrigatoriamente dentro** do construtor do Todo (ou clonadas de forma profunda caso sejam passadas).

O construtor do Todo pode receber configurações básicas primitivas (ex: `temperaturaMaxima`, `precisao`), mas ele mesmo executa os `new` das suas Partes. Uma Parte não deverá ser compartilhada arbitrariamente entre diferentes Todos nem ser tratada pelo restante do sistema como uma entidade independente.

```java
public class EntidadePrincipal {
    // Relacoes 1:1 e 1:N
    private ComponenteA parteUm;
    private ComponenteB parteDois;
    private List<ComponenteC> partesMultiplas;

    // O Todo instancia suas partes. O ciclo de vida eh estrito.
    public EntidadePrincipal(String id, double configBase, int qtdePartesC) {
        this.id = id;
        this.parteUm = new ComponenteA(configBase);
        this.parteDois = new ComponenteB();
        this.partesMultiplas = new ArrayList<>();

        for (int i = 0; i < qtdePartesC; i++) {
            this.partesMultiplas.add(new ComponenteC());
        }
    }
}
```

Listing 1: Exemplo Genérico de Fase de Construção Segura

3.3 Cardinalidade

O Todo deverá possuir pelo menos:

- uma relação **1:1** com uma Parte;
- uma relação **1:N** com uma Parte.

Por exemplo:

```
Máquina
├── 1 Motor
├── 1 SensorDeTemperatura
└── 1..* Atuadores
```

A cardinalidade deverá ser coerente com o domínio. Para relações com múltiplas instâncias, deverão ser utilizadas estruturas de dados adequadas. A escolha deverá considerar as características da relação representada.

3.4 Composição versus Agregação

Até a Tarefa 3, as unidades de produção ficavam possivelmente soltas em uma lista no Gerenciador. Agora, elas devem pertencer a um objeto Agrupador.

Requisitos do Agrupador:

- Crie uma classe intermediária para agrupar instâncias do seu Todo (ex: `LinhaDeProducao`, `SetorDeUsinagem`, `BancadaPrincipal`).
- Essa classe deve gerenciar uma coleção (1:N) das entidades produtivas.
- **Cópia Defensiva:** Se o agrupador possuir um método `getEntidades()`, ele **não deve retornar a lista original** para evitar que classes externas manipulem a estrutura. Utilize `Collections.unmodifiableList()` ou devolva um clone da lista.

O Agrupador deverá manter uma coleção de unidades produtivas. Essa relação **não precisa ser uma composição**.

A dupla deverá analisar o domínio e determinar se o ciclo de vida das unidades produtivas realmente depende do Agrupador. Por exemplo, se uma `Maquina` puder ser retirada de uma `LinhaDeProducao` e posteriormente adicionada a outra linha, então é possível que a relação seja melhor representada como agregação ou associação:

```
LinhaDeProducao ◇—— 1..* Maquina
```

enquanto a relação entre uma máquina e seus componentes pode ser:

```
Maquina ♦——— 1 Motor
Maquina ♦——— 1 Sensor
Maquina ♦——— 1..* Atuador
```

O objetivo é que a dupla compreenda que nem toda relação "possui" entre duas classes representa composição. A escolha da relação deverá ser coerente tanto com o domínio quanto com o ciclo de vida dos objetos.

3.5 Aplicação da Lei de Deméter (*Tell, Don't Ask*)

O sistema externo (como o `GerenciadorProducao` ou a interface do usuário) **nunca** deve recuperar uma Parte interna para invocar um método nela. O Todo deve expor métodos de alto nível que delegam a ação.

```
// ERRADO (Viola Demeter e vaza encapsulamento da parte interna)
entidadePrincipal.getComponenteA().iniciarProcesso();

// CORRETO (Delega a responsabilidade)
entidadePrincipal.processarDemanda();
// Internamente, a entidadePrincipal eh quem chama this.parteUm.iniciarProcesso()
```

Listing 2: Evitando Dependências - Lei de Deméter

Se alguma funcionalidade exigir leitura de dados de uma Parte (ex: ler a temperatura atual), o Todo deve prover um método repassador (ex: `entidadePrincipal.lerTemperaturaAtual()`).

O objetivo é que o código cliente precise conhecer apenas o Todo, e não sua estrutura interna.

3.6 Proteção das Coleções

As coleções utilizadas para armazenar Partes ou unidades produtivas não deverão ser expostas diretamente.

```
// ERRADO (Permite que o codigo externo modifique diretamente a estrutura interna)
public List<Maquina> getMaquinas() {
    return maquinas;
}

// CORRETO (Colecao imutavel)
public List<Maquina> getMaquinas() {
    return Collections.unmodifiableList(maquinas);
}
```

Listing 3: Protegendo as Coleções

O objetivo é garantir que alterações na estrutura interna sejam realizadas exclusivamente pelas operações oferecidas pelo objeto responsável pela coleção.

3.7 Auditoria em Cascata

Na Tarefa 3, foi introduzida a interface `Auditavel`. Com a nova arquitetura composta, a auditoria deve se propagar em cascata (Agrupador → Todo → Partes).

1. Quando o sistema pedir o `gerarRelatorioDiagnostico()` do Agrupador, ele chama o relatório de cada Todo.

2. O método `gerarRelatorioDiagnostico()` do Todo deve invocar métodos de checagem em todas as suas Partes.

3. O resultado final exibido no terminal deve ser uma string hierárquica e indentada mostrando a saúde estrutural completa (Ex: *Setor A > Máquina 1 > Sensor: OK | Motor: Superaquecido*).

Não é obrigatório que todas as Partes implementem `Auditavel`. A dupla deverá avaliar quais componentes possuem estado ou comportamento relevante para diagnóstico. Uma Parte puramente estrutural, que não possua condição operacional própria, não precisa ser artificialmente transformada em um componente auditável.

3.8 Diagrama UML da Estrutura

A dupla deverá elaborar um **diagrama de classes UML** representando as relações estruturais introduzidas nesta tarefa.

O diagrama deverá apresentar, no mínimo:

- o Agrupador;
- o Todo;
- as três Partes;
- os relacionamentos entre essas classes;
- as cardinalidades;
- as relações de composição;
- a relação de agrupamento, quando aplicável;
- os principais atributos e métodos necessários para compreender a estrutura.

As relações de composição deverão utilizar a notação UML apropriada.

O diagrama deverá ser coerente com o código implementado. Não serão aceitas relações de composição ou agregação representadas no UML que não correspondam à arquitetura efetivamente implementada.

Entrega do diagrama O diagrama deverá ser entregue juntamente com o código. São aceitos, por exemplo, arquivos `.pdf`, `.png` ou `.jpg`.

Recomenda-se também o envio do arquivo-fonte editável utilizado para criar o diagrama, como `.drawio`, quando aplicável.

3.9 Menu do Sistema Atualizado

O menu principal deve ser adaptado para exibir a nova hierarquia do seu domínio. Mantenha os submenus, mas inclua opções para interagir com os Agrupadores e visualizar a auditoria estrutural detalhada.

4 Criatividade e Evolução do Tema

Sendo o tema da fábrica livre, a **coesão da sua modelagem será rigorosamente avaliada**.

- **Nomes e Funções Temáticas:** Suas Partes devem ter funções lógicas. Se você criou uma `Cervejaria`, não chame a parte de `Componente1`. Chame de `ValvulaDePressao` ou `FiltroDeMosto`.
- **Comportamentos Independentes:** Demonstre que as Partes importam. Uma Parte quebrada deve afetar o desempenho do Todo (ex: se o `Filtro` falha, a `Qualidade` do produto final daquela linha de produção cai).

5 Avaliação

A avaliação levará em conta os seguintes critérios:

1. **Modelagem de Composição Temática e UML (3,0 pts)**
   - Diferenciação clara e lógica entre o Agrupador, o Todo e as Partes de acordo com o tema;
   - Aplicação de Composição Estrita (as partes são destruídas junto com o Todo);
   - Existência de pelo menos 3 componentes internos distintos por unidade produtiva;
   - Diagrama UML coerente com a implementação;
   - Utilização correta da notação UML para composição e cardinalidade.
2. **Fases de Construção e Instanciação (2,0 pts)**
   - Partes de composição inicializadas pelo Todo;
   - Todo construído em estado válido;

- controle adequado do ciclo de vida das Partes;
- ausência de dependência desnecessária de objetos externos para a construção das Partes;
- arquitetura coerente com a relação de ownership representada.

3. **Desacoplamento e Lei de Deméter (2,0 pts)**
   - Ausência de navegação profunda (*Train Wreck*) e vazamento de referências;
   - Partes não expostas diretamente;
   - Utilização adequada de métodos de delegação;
   - Aplicação de cópias defensivas ou coleções imutáveis.
4. **Comportamento e Auditoria (1,5 pts)**
   - Comportamento das Partes coerente com o domínio;
   - Delegação de comportamentos (como a Auditoria) fluindo corretamente do nível mais alto até as Partes mais baixas da hierarquia.
5. **Integração de Sistemas Anteriores (1,5 pts)**
   - Manutenção das funcionalidades da Tarefa 4 (Gradle e Tratamento de Exceções aplicados à nova arquitetura);
   - Menu atualizado e coerente com a nova estrutura;
   - Código organizado.
6. **Penalidade:** Não compilação via Gradle = -5,0 pontos.

6 Entrega

1. **GitHub:** Release denominada `tarefa5-RA1-RA2` contendo o código fonte atualizado e o diagrama UML;
2. **Google Forms:** Formulário preenchido com nomes, RAs e link direto da release no GitHub.

Atenção: Crie a release primeiro, copie o link e, em seguida, submeta no Forms.
O prazo final de submissão é o dia **22/10/2026 às 23h59**. Entregas atrasadas sofrerão penalização de **25% sobre a nota obtida**. Não serão aceitas entregas efetuadas após **29/10/2026**.

6.1 Compilação e Execução

O projeto deve continuar compilando e executando utilizando a estrutura do Gradle:

```
./gradlew build
./gradlew run
```

7 Dicas de Boas Práticas

- **Composição não é apenas "ter um atributo":** avalie se a Parte realmente pertence ao Todo e se seu ciclo de vida é controlado por ele.
- **Composição versus agregação:** se um objeto puder existir e ser transferido entre diferentes estruturas independentemente, questione se a relação realmente deve ser uma composição.
- **Collections.unmodifiableList():** A melhor forma de retornar uma coleção interna sem permitir que quem a chamou adicione ou remova itens indevidamente.
- **Fachadas Internas (*Facades*):** Pense no "Todo" como um gerente. O gerente não executa a tarefa manualmente, ele sabe qual funcionário ("Parte") acionar. O sistema só precisa falar com o gerente.
- **Responsabilidade Única (SRP):** A parte interna (ex: Filtro) não deve saber sobre Orçamento ou Estratégias de Produção. Ela tem apenas um trabalho: filtrar, desgastar-se ou quebrar. Quem entende das regras de negócio maiores são os níveis superiores (Todo e Agrupador).